FDE SCENARIO DRILLS #02

FDE SCENARIO DRILLS · ISSUE 02 OF 08

# Forty pods, zero database connections.

Scenario: autoscaling turned a traffic spike into an outage.

02

AR

**Akhilesh Ranjan Singh**

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

# How it unfolded

1. Traffic spike
2. HPA: 4 → 40 pods
3. 40 pools × 20 connections
4. max_connections exceeded
5. Every new request fails

AR
Akhilesh Ranjan Singh
github.com/iarsingh

# Stabilise

01 **Cap replicas**

Lower HPA max to what the database can serve.

02 **Shrink pools**

Fewer connections per pod, shorter idle timeout.

03 **Shed load**

Return 503 with Retry-After instead of queueing forever.

AR
**Akhilesh Ranjan Singh**
github.com/iarsingh

# Add a pooler

| CLOUD | OPTION |
| --- | --- |
| **AWS** | RDS Proxy or PgBouncer |
| **GCP** | PgBouncer, or managed pooling where available |
| **Azure** | Built-in PgBouncer on flexible server |

AR
**Akhilesh Ranjan Singh**
github.com/iarsingh

THE LESSON

# Scale the app and the database together.

Every replica is also a set of database connections. Budget them.

AR
**Akhilesh Ranjan Singh**
github.com/iarsingh

# Prevent it

- [x] Connection budget per service
- [x] HPA max tied to that budget
- [x] Load test at month-end volume
- [x] Alert at 80% of max connections
- [x] Runbook for connection storms

AR
**Akhilesh Ranjan Singh**
github.com/iarsingh

FDE SCENARIO DRILLS #02

KEY TAKEAWAY

# Every pod you add is also a set of database connections.

→ Cap autoscaling to what dependencies can take

→ Put a pooler in front of Postgres

→ Load test the real peak

YOUR TURN

**What is the dependency your autoscaler forgets about?**

FOLLOW AKHILESH RANJAN SINGH

**Next in the series: #03 The pod that could not reach the LLM**

Save · Repost

AR
Akhilesh Ranjan Singh
github.com/iarsingh